

PROJECT: INVESTIGATE A MALWARE-INFECTED SYSTEM

LOGS AND REPORT FINDINGS AND SOLUTIONS

Security Incident Report

Incident ID: BT-IR-20240401

Date: June 25, 2025

Prepared for: BrightTech Cyber Security Team

Prepared by: Incident Response Analyst

1. Executive Summary of The Event

On April 1, 2024, internal host 192.168.50.23 was identified as compromised. Indicators included unauthorized outbound connections, blocked login attempts, and the presence of malicious files. Behavioral analysis and log data confirmed the presence of both a **Trojan Downloader** and an **Autorun Worm**. Immediate containment was initiated.

Timeline of Events

Timestamp	Event Description
14:45:00	Outbound connection to 203.0.113.5:22 (SSH) from 192.168.50.23
14:50:30	Outbound connection to 192.0.2.89:9090 from same host
15:12:00	Blocked inbound login attempt from 198.51.100.10 due to repeated failures
15:30:00	Malware detected: Worm.Autorun.Script in C:\Users\Guest\Documents\hidden.vbs
16:00:00	Deep scan revealed two threats; both files quarantined

1. Logs & Evidence Collected

Network Logs

Timestamp: 2024-04-01 14:45:00

Source IP: 192.168.50.23

Destination IP: 203.0.113.5

Port: 22

Protocol: TCP

Status: ALLOWED

Timestamp: 2024-04-01 14:50:30

Source IP: 192.168.50.23

Destination IP: 192.0.2.89

Port: 9090

Protocol: TCP

Status: ALLOWED

Timestamp: 2024-04-01 15:12:00

Source IP: 198.51.100.10

Destination IP: 192.168.50.23

Action: BLOCKED

Reason: Multiple failed authentication attempts

Malware Logs

Timestamp: 2024-04-01 15:30:00
Threat Name: Worm.Autorun.Script
File Path: C:\Users\Guest\Documents\hidden.vbs
Action Taken: Suspended
Detection Method: Behavior Analysis

Timestamp: 2024-04-01 16:00:00
Total Files Scanned: 200,000
Threats Detected: 2
- C:\ProgramData\startup.bat (Trojan Downloader)
- C:\Users\Guest\Documents\hidden.vbs (Worm)
Action Taken: Quarantined, Further Investigation Required

Windows Event Logs

Event ID	Description	User/Process
4672	Special privileges assigned	UnknownUser
4776	Credential validation attempt (linked to brute-force)	SYSTEM / Source: 198.51.100.10
7034	Windows Update Service terminated unexpectedly	Suspected DLL injection

Impact Assessment

Area	Impact Summary
System Integrity	Confirmed malware infection & unauthorized script execution
Confidentiality	Potential for credential or data exfiltration
Persistence Risk	Trojan embedded in startup file; possible backdoor remains
Policy Violation	Use of Guest account with malware in personal documents folder
Operational Risk	Infected system could spread worm to connected drives or systems

2. TABLE OF INDICATORS OF COMPROMISE (IOCs)

Ioc Indicator type	Description	Evidence Collected
--------------------	-------------	--------------------

203.0.113.5:22	Outbound SSH connection to reserved/test IP	Network log (2024-04-01 14:45:00) from 192.168.50.23 to 203.0.113.5:22
192.0.2.89:9090	Outbound connection to suspicious port commonly used for admin/web panels	Network log (2024-04-01 14:50:30) from 192.168.50.23
198.51.100.10	Repeated failed login attempts from external IP (potential brute-force)	Firewall log (2024-04-01 15:12:00) – Blocked with “Multiple failed authentication”
C:\Users\Guest\Documents\hidden.vbs	Worm script using autorun functionality	Detected as Worm.Autorun.Script (15:30:00); action: Suspended & quarantined
C:\ProgramData\startup.bat	Trojan downloader designed to run at startup	Detected in full scan (16:00:00); action: Quarantined
Event ID 4672	Special privileges assigned to unknown user (possible privilege escalation)	Security event log: "User: UnknownUser", "Privilege: SeAssignPrimaryTokenPrivilege"
Event ID 4776	Authentication attempts by SYSTEM user (potentially brute-forced credentials)	Event log source: 198.51.100.10 → SYSTEM
Event ID 7034	Windows Update Service terminated unexpectedly (likely due to injected code)	System log (service crash, faulting module: injected.dll)

3. Summary of Findings and Analysis

a. Compromised Internal Host

The system with IP address 192.168.50.23 OS: Windows 10 was identified as the central point of compromise. It exhibited behaviors consistent with both malware infection and external probing.

b. Unauthorized Outbound Network Activity

- The host made **outbound connections** to **non-routable/test IP addresses**:
 - 203.0.113.5 over **SSH (port 22)**
 - 192.0.2.89 over **port 9090**
- These addresses are typically reserved and not routable in public networks, indicating:
 - Malware attempting to contact Command & Control (C2)
 - Use of hardcoded testing IPs, or
 - Possible misdirection by an attacker

c. Brute-Force Attack Attempted

- Multiple **failed login attempts** from external IP 198.51.100.10 were recorded.
- These were **blocked** by existing security controls but confirm that the host was being **actively targeted**.

d. Malware Discovered

Two malware files were discovered on the host:

- hidden.vbs – **Worm.Autorun.Script**
 - Found in the Guest user's documents folder
 - Indicates propagation via removable devices or user downloads
- startup.bat – **Trojan Downloader**

- Located in the C:\ProgramData startup directory
- Designed to **download further payloads** upon boot or login

Both were quarantined during the investigation.

e. Privilege Escalation and Instability

- **Event ID 4672:** Showed **special privileges being assigned** to an unknown user
- **Event ID 4776:** Validated that credential attempts were being made, potentially brute-force
- **Event ID 7034:** Indicated **unexpected service terminations**, likely due to injected or unstable code (possibly malware DLLs)

Conclusively, the investigation revealed a **multi-stage compromise**:

- Initial infection via autorun or startup script
- Persistence through startup modification
- Outbound traffic suggesting communication with attacker-controlled infrastructure
- Inbound brute-force attempts
- Confirmed malware: **worm and trojan downloader**

The host is compromised and poses a risk to other systems. Immediate containment and a broader network sweep are warranted.

4. Recommended Remediation Actions

Action	Status
Immediately isolate affected host (192.168.50.23)	✓ Done
Quarantine and analyze hidden.vbs, startup.bat	✓ Done
Perform memory and disk imaging for forensic use	Pending
Conduct full malware removal and reimage host	Pending
Reset passwords for all users on host	Pending

Perform network sweep using PowerShell IOC script	In Progress
Implement tighter egress firewall rules	Pending
Block RFC 5737/reserved test IP ranges	Pending

This is a **confirmed multi-vector security incident** involving malware infection, privilege assignment abuse, and unauthorized network activity. The situation is **contained** at the host level, but **further investigation** is required to confirm:

- Scope of lateral movement
- Any additional compromised systems
- Root cause and entry vector

5A. Lessons Learned from the Incident

1. Weak Perimeter Controls Enabled Outbound Threat Activity

- Malware on the internal host was able to make **unauthorized outbound connections** to reserved/test IPs over ports like **22 (SSH)** and **9090**.
- **Lesson:** Implement **strict egress filtering** on firewalls to block unusual outbound traffic, especially to non-routable/reserved IPs and high-risk ports.

2. Lack of Malware Containment Enabled Persistence

- The **Trojan (startup.bat)** was embedded in a common system-wide **startup directory** and could persist through reboots.
- **Lesson:** Enforce **startup/script folder monitoring**, and use **application whitelisting** to prevent execution of unauthorized batch/VBS scripts.

3. Endpoint Protections Detected Late-Stage Behavior

- The worm (hidden.vbs) and trojan were only detected after **behavioral triggers**, not during initial infection.
- **Lesson:** Use **real-time file scanning**, **heuristic analysis**, and **sandboxing** to catch malware **before execution**, not just after.

4. User Account Misuse Went Unnoticed

- Malware was found under the Guest profile — a low-privilege account that **should be disabled**.
- **Lesson:** Audit and disable all unnecessary or default accounts. Apply the **principle of least privilege** rigorously.

5. Suspicious Event Logs Were Missed

- Privileged logon (Event ID 4672), credential validation (4776), and a system crash (7034) **were not correlated** until after malware detection.
- **Lesson:** Set up **SIEM alerts** for critical security events, and establish a **baseline of normal behavior** for anomaly detection.

6. No Early Detection of Lateral Movement

- The worm could have spread via USB or shared drives — no prior indicators suggested **lateral propagation** was being monitored.
- **Lesson:** Deploy **network-wide IOC scanning** tools and conduct **regular threat hunts** to detect stealthy movement.

7. Response Was Reactive, Not Proactive

- Action was only taken after full compromise and multiple red flags.
- **Lesson:** Develop and test an **incident response plan**, including isolation procedures, logging policy, and staff training for early threat detection.

5B. Based on the findings of the incident, here are targeted **preventive measures** to reduce the likelihood of recurrence and strengthen BrightTech's security posture:

Recommended Preventive Measures

1. Endpoint Protection & Hardening

- **Install and maintain updated antivirus/EDR solutions** with real-time protection
- **Disable unused accounts** like Guest and enforce **least privilege** on all users

- Restrict script execution (e.g., .vbs, .bat) through:
 - Group Policy (Software Restriction Policies)
 - Application whitelisting (e.g., Windows Defender Application Control)

2. Network Security Enhancements

- **Enforce outbound firewall rules:**
 - Block unnecessary ports (e.g., 22, 9090)
 - Disallow traffic to test/reserved IPs (e.g., 203.0.113.0/24, 192.0.2.0/24)
- Implement **intrusion detection/prevention systems (IDS/IPS)**
- **Segment the network** to contain lateral movement (e.g., guest devices vs. core infrastructure)

3. Advanced Threat Detection

- Deploy a **SIEM (Security Information and Event Management)** tool to:
 - Correlate events like 4672 (privilege escalation), 4776 (login attempts), and 7034 (service failures)
 - Raise alerts for anomalous behavior or unauthorized privilege assignments
- Perform **weekly IOC sweeps** using scripts (like the PowerShell scan you ran)

4. File & Script Monitoring

- Monitor and audit:
 - C:\ProgramData\

- User profile folders (e.g., Documents, Downloads)
- Startup directories
- Use **file integrity monitoring (FIM)** tools to detect unauthorized changes to critical paths

5. Patch & Update Management

- Ensure **timely OS and third-party software updates**
- Automate patch management using tools like WSUS, SCCM, or cloud alternatives

6. User Awareness & Access Control

- Train employees to identify:
 - Phishing attempts
 - Suspicious file downloads (e.g., USB autorun scripts)
- Enforce:
 - **Multi-Factor Authentication (MFA)**
 - Strong, unique passwords
 - Role-based access control (RBAC)

7. Policy & Response Readiness

- Maintain and test:
 - **Incident Response Plan (IRP)**
 - **Backup and Disaster Recovery Plan**

- Regularly review and update:
 - **Acceptable Use Policies**
 - **Remote Access Policies**

8. Removable Media Controls

- Disable autorun features via Group Policy
- Use endpoint control solutions to **limit USB access** to authorized devices

9. Continuous Monitoring & Audit

- Monitor:
 - Logon events (especially from SYSTEM or unknown users)
 - Unusual service terminations
- Enable **centralized log collection** and **retention** for at least 90 days

10. Red Team / Blue Team Exercises

- Regularly conduct **penetration tests** and **tabletop exercises**
- Simulate scenarios like:
 - Worm propagation
 - Privilege escalation
 - C2 communication detection

```
Administration: Windows PowerShell ISE (v8.0)
File Edit View Tools Debug Add-ons Help

PS C:\Windows\system32> # PowerShell Script to Extract Indicators of Compromise from Windows Security Log

# Define output path
$OutputPath = "$env:USERPROFILE\Desktop\SuspiciousEvents.csv"

# Create a list to store results
$results = @()

Write-Output "Searching Event Log for suspicious activity..."

# 1. Event ID 4672 - Special Privilege Assigned to New Logon
$events4672 = Get-WinEvent -FilterHashtable @{'LogName'='Security'; ID=4672} -MaxEvents 1000
foreach ($event in $events4672) {
    $xml = [xml]$event.ToXml()
    $saccount = $xml.Event.EventData.Data | Where-Object { $_.Name -eq 'SubjectUserName' } | Select-Object -ExpandProperty Name
    $privileges = $xml.Event.EventData.Data | Where-Object { $_.Name -eq 'PrivilegeList' } | Select-Object -ExpandProperty Name

    if ($privileges -like "SeBackupPrimaryTokenPrivilege" -or $saccount -eq "UnknownUser") {
        $results += [PSCustomObject]@{
            TimeCreated = $event.TimeCreated
            EventID = 4672
            Username = $saccount
            Privileges = $privileges
            Description = "Special privileges assigned to logon"
        }
    }
}

# 2. Event ID 4776 - Credential Validation Attempt
$events4776 = Get-WinEvent -FilterHashtable @{'LogName'='Security'; ID=4776} -MaxEvents 1000
foreach ($event in $events4776) {
    $xml = [xml]$event.ToXml()
    $saccount = $xml.Event.EventData.Data | Where-Object { $_.Name -eq 'TargetUserName' } | Select-Object -ExpandProperty Name
    $sip = $xml.Event.EventData.Data | Where-Object { $_.Name -eq 'Workstation' } | Select-Object -ExpandProperty Name

    if ($saccount -eq "SYSTEM" -or $sip -eq "198.51.100.10") {
        $results += [PSCustomObject]@{
            TimeCreated = $event.TimeCreated
            EventID = 4776
            Username = $saccount
            SourceIP = $sip
            Description = "SYSTEM credential validation from suspicious IP"
        }
    }
}

# 3. Event ID 1015 (Optional: Application Crash, e.g., explorer.exe)
$events1015 = Get-WinEvent -FilterHashtable @{'LogName'='Application'; ID=1015} -MaxEvents 1000
foreach ($event in $events1015) {
    if ($event.Message -like "injected.dll" -and $event.Message -like "explorer.exe") {
        $results += [PSCustomObject]@{
            TimeCreated = $event.TimeCreated
            EventID = 1015
            Process = "explorer.exe"
            Module = "injected.dll"
            Description = "Explorer crash due to suspicious DLL"
        }
    }
}

# Export the results to CSV
if ($results.Count -gt 0) {
    $results | Export-Csv -Path $OutputPath -NoTypeInformation
    Write-Output "`n[n] IOC findings exported to: $OutputPath"
} else {
    Write-Output "`n[n] No suspicious activity found in the scanned logs."
}

Searching Event Log for suspicious activity...
```

```
Administration: Windows PowerShell ISE (v8.0)
File Edit View Tools Debug Add-ons Help

TimeCreated = $event.TimeCreated
EventID = 4776
Username = $saccount
SourceIP = $sip
Description = "SYSTEM credential validation from suspicious IP"
}
}

# 3. Event ID 1015 (Optional: Application Crash, e.g., explorer.exe)
$events1015 = Get-WinEvent -FilterHashtable @{'LogName'='Application'; ID=1015} -MaxEvents 1000
foreach ($event in $events1015) {
    if ($event.Message -like "injected.dll" -and $event.Message -like "explorer.exe") {
        $results += [PSCustomObject]@{
            TimeCreated = $event.TimeCreated
            EventID = 1015
            Process = "explorer.exe"
            Module = "injected.dll"
            Description = "Explorer crash due to suspicious DLL"
        }
    }
}

# Export the results to CSV
if ($results.Count -gt 0) {
    $results | Export-Csv -Path $OutputPath -NoTypeInformation
    Write-Output "`n[n] IOC findings exported to: $OutputPath"
} else {
    Write-Output "`n[n] No suspicious activity found in the scanned logs."
}

Searching Event Log for suspicious activity...
```

```
Administration: Windows PowerShell ISE (v8.0)
File Edit View Tools Debug Add-ons Help

PS C:\Windows\system32> # PowerShell Script to Investigate Windows Update Service (wuauserv)

$OutputPath = "$env:USERPROFILE\Desktop\UpdateServiceAnalysis.csv"
$results = @()

Write-Output "`n[n] Checking Windows Update Service Status..."

# 1. Check if wuauserv is running
$service = Get-Service -Name "wuauserv" -ErrorAction SilentlyContinue
if ($service) {
    $results += [PSCustomObject]@{
        Check = "Service Status"
        Result = $service.Status
    }
    $results += [PSCustomObject]@{
        Check = "Service StartType"
        Result = (Get-WmiObject -Class Win32_Service -Filter "Name='wuauserv'").StartMode
    }
} else {
    $results += [PSCustomObject]@{
        Check = "Service Check"
        Result = "Windows Update Service (wuauserv) Not Found"
    }
}

# 2. Check for Service crash logs (Event ID 7034)
Write-Output "`n[n] Searching Event Logs for service crashes..."
$events7034 = Get-WinEvent -FilterHashtable @{'LogName'='System'; ID=7034} -MaxEvents 1000
foreach ($event in $events7034) {
    $results += [PSCustomObject]@{
        Check = "Crash Detected"
        Result = "Windows Update service terminated unexpectedly"
        Time = $event.TimeCreated
    }
}
```

```

Administrator: Windows PowerShell ISE (x86)
File Edit View Tools Debug Add-ons Help

Check = "Crash Detected"
Result = "Windows Update service terminated unexpectedly"
Time = $event.TimeCreated
}
}

# 3. Check Registry for tampering (Disabling Windows Update)
Write-Output "n# Checking Registry for update policy modifications..."
$regPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU"
if (Test-Path $regPath) {
    $wuPolicy = Get-ItemProperty -Path $regPath
    foreach ($property in $wuPolicy.PSObject.Properties) {
        $results += [PSCustomObject]@{
            Check = "Registry Policy: $($property.Name)"
            Result = $property.Value
        }
    }
} else {
    $results += [PSCustomObject]@{
        Check = "Registry Policy Check"
        Result = "No group policy settings found under WindowsUpdate\AU"
    }
}

# 4. Export all findings
$results | Export-Csv -Path $OutputPath -NoTypeInformation
Write-Output "n# Analysis complete. Results saved to: $OutputPath"

Checking Windows Update Service Status...
Searching Event Logs for service crashes...
Checking Registry for update policy modifications...
# Analysis complete. Results saved to: C:\Users\USER\Desktop\UpdateService_Analysis.csv

PS C:\Windows\system32> # PowerShell Script to Investigate Windows Update Service (wuauclt)

```

G11						
	A	B	C	D	E	F
1	ComputerName	HiddenVBS	StartupBAT	Event_4672	Event_7034_ServiceCrashes	Status
2	192.168.50.20	Unknown	Unknown	N/A	N/A	Offline/No Ping
3	192.168.50.21	Unknown	Unknown	N/A	N/A	Offline/No Ping
4	192.168.50.23	Unknown	Unknown	N/A	N/A	Offline/No Ping
5	192.168.50.24	Unknown	Unknown	N/A	N/A	Offline/No Ping
6						
7						
8						
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						
19						
20						
21						
22						
23						

```

Administrator: Windows PowerShell ISE (x86)
File Edit View Tools Debug Add-ons Help

StartupBAT = if ($trojan) { 'FOUND' } else { 'Not Found' }
Event_4672_4776_Count = $eventSummary
Event_7034_ServiceCrashes = $crashes.Count
Status = "OK"
}
} else {
    $results += [PSCustomObject]@{
        ComputerName = $computer
        HiddenVBS = "Unknown"
        StartupBAT = "Unknown"
        Event_4672_4776_Count = "N/A"
        Event_7034_ServiceCrashes = "N/A"
        Status = "Offline/No Ping"
    }
}
} catch {
    $results += [PSCustomObject]@{
        ComputerName = $computer
        HiddenVBS = "Error"
        StartupBAT = "Error"
        Event_4672_4776_Count = "Error"
        Event_7034_ServiceCrashes = "Error"
        Status = "Error: $_"
    }
}
}

# Export results
$results | Export-Csv -Path $logPath -NoTypeInformation
Write-Host "n# Scan complete. Results saved to: $logPath" -ForegroundColor Green

Scanning 192.168.50.20...
Scanning 192.168.50.21...
Scanning 192.168.50.23...
Scanning 192.168.50.24...

# Scan complete. Results saved to: C:\Users\USER\Desktop\Malware_IDC_Scan_Results.csv

```